Autoguided Online Data Curation for Diffusion Model Training

Valeria Pais
University of Glasgow
11 Chapel Ln, Glasgow G11 6EW, UK
`v.pais-malacalza.1@research.gla.ac.uk`

Luis Oala
Dotphoton
Nordstrasse 3, 6300 Zug, Switzerland
`luis.oala@dotphoton.com`

Daniele Faccio
University of Glasgow
11 Chapel Ln, Glasgow G11 6EW, UK
`daniele.faccio@glasgow.ac.uk`

Marco Aversa
Dotphoton
Nordstrasse 3, 6300 Zug, Switzerland
`marco.aversa@outlook.com`

Abstract

The costs of generative model compute rekindled promises and hopes for efficient data curation. In this work, we investigate whether recently developed autoguidance and online data selection methods can improve the time and sample efficiency of training generative diffusion models. We integrate joint example selection (JEST) and autoguidance into a unified code base for fast ablation and benchmarking. We evaluate combinations of data curation on a controlled 2*-D synthetic data generation task as well as* (3×64^2)*-D image generation. Our comparisons are made at equal wall-clock time and equal number of samples, explicitly accounting for the overhead of selection. Across experiments, autoguidance consistently improves sample quality and diversity. Early AJEST —applying selection only at the beginning of training— can match or modestly exceed autoguidance alone in data efficiency on both tasks. However, its time overhead and added complexity make autoguidance or uniform random data selection preferable in most situations. These findings suggest that while targeted online selection can yield efficiency gains in early training, robust sample quality improvements are primarily driven by autoguidance. We discuss limitations and scope, and outline when data selection may be beneficial.*

1. Introduction

Scaling trends have made modern generative modeling increasingly expensive, shifting attention from model- and hardware-centric efficiency towards data efficiency (e.g., [3, 5, 16, 24, 31]). Evidence suggests that targeted curation can improve performance by removing redundancy and noise [10]. Diffusion models, however, aim to approximate full data distributions, raising the question of whether data selection can help without harming diversity; recent work indicates pruning can help in some regimes [4].

We study online data selection for diffusion models through Autoguided JEST (AJEST), which integrates joint example selection (JEST) with autoguidance. We evaluate on a controlled 2-D task and Tiny ImageNet at $3 \times 64 \times 64$. Comparisons are made at equal wall-clock time and equal number of samples, explicitly accounting for selection overhead. Our focus is practicality: does targeted early selection provide efficiency gains, and how does it interact with autoguidance? **Our contributions and findings** are as follows:

1. A unified code base to run combinations of autoguidance and JEST for online data curation of diffusion models[1]
2. An evaluation harness for time and data efficiency of data curation during diffusion model training.
3. Early AJEST can match or modestly exceed autoguidance in data efficiency on both a 2-D and (3×64^2)-D; however, its time overhead and added complexity makes it less attractive for most applications.
4. A transparent discussion of limitations and where data curation hits limits.

Related work. Data selection and active learning span diversity-based approaches (e.g., k-center and submodular selections [1, 29]) and gradient/importance-based methods [2, 22, 28, 30, 37], as well as instance difficulty and loss-based strategies [6, 9, 11, 15, 27, 34, 35]. However, existing literature also suggests that random data selection can outperform these approaches in many settings [25].

JEST [8] formalizes online joint selection using a learner–reference pair and has been shown capable of replacing knowledge distillation in contrastive setups [36]. For diffusion models, classifier-free guidance [13] improves fidelity at the cost of diversity, whereas autoguidance [18]

---

[1] `https://github.com/0xInfty/Autoguided`

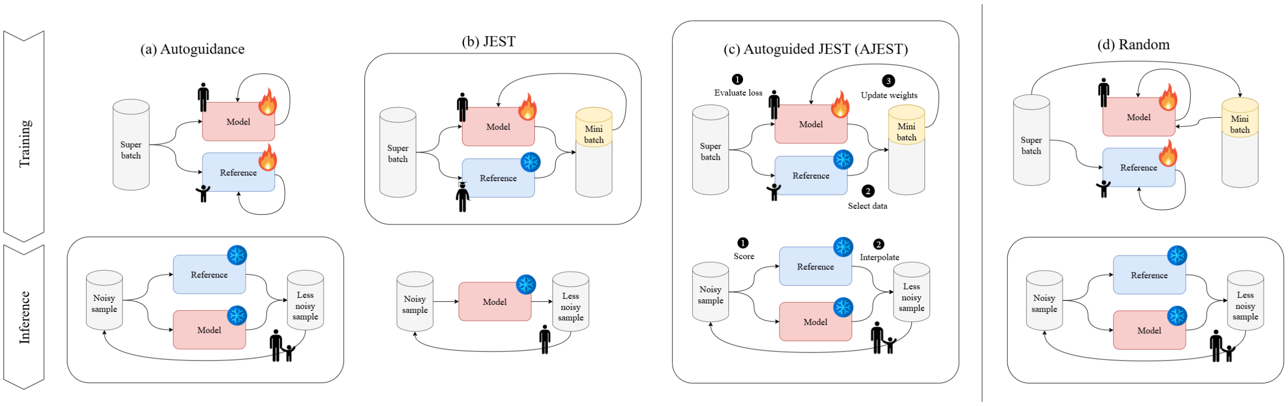

Figure 1. Visual summary. (a) JEST: a large pre-trained foundational reference model is used to select data to train the main model, which can be smaller in size; inference is done using only the main model. (b) Autoguidance: a smaller reference model is trained for less iterations; both reference and main model score samples during the denoising process. (c) Autoguided JEST (AJEST): combination of JEST's training strategy and autoguidance's small less-trained reference and collaborative denoising process. (d) Random: JEST is replaced with uniform random data selection to act as a benchmarking baseline with independent training for the main model.

leverages a weaker guide to boost quality while better preserving coverage. Our work examines how JEST-style online selection interacts with autoguidance for diffusion models under equal-time and equal-backprop comparisons.

2. Methods

We briefly review the core ingredients underlying the experiments, but we include details in Appendix A.

Diffusion models. Diffusion models learn a family of scores over noise levels and iteratively denoise samples from Gaussian noise towards the data distribution [17]. Sampling follows a probability-flow ODE discretized over a noise (time) schedule; the model provides score evaluations that guide each denoising step. We adopt EDM/EDM2 conventions for preconditioning and training [19].

Given the cost and scale of diffusion training, online data curation can appear appealing to reduce redundancy while preserving performance. Here we evaluate three regimes: an online learner–reference strategy (JEST) that adapts to training dynamics in combination with autoguidance (AJEST), a standalone autoguidance baseline, and a cheap random subset selection baseline (see Fig. 1).

JEST. Joint Example Selection (JEST) [8] pairs a learner with a reference model to score examples in a super-batch and draws a mini-batch via softmax sampling over learnability scores. With super-batch size B and filtering ratio f, the update mini-batch has size $b = (1 - f)B$. The canonical learnability contrasts learner and reference losses and prioritizes examples that are easy to learn for the reference but not for the learner. JEST can act as implicit knowledge distillation in contrastive setups [36]. Exact scoring, chunked sampling logits, and our stability normalization are given in Appendix A.1.

Autoguidance. Classifier-free guidance (CFG) [13] improves fidelity at a diversity cost by mixing conditional/unconditional scores. Autoguidance [18] instead leverages a weaker guide model (smaller or less-trained) to produce a corrective signal that boosts quality while better preserving coverage; we follow their formulation and report both unguided and autoguided sampling. Exact guiding signal and collaborative sampling details are in Appendix A.1.

AJEST. We integrate JEST with autoguidance for diffusion models. A smaller guide (weaker version of the learner) serves as the JEST reference. Because the learner quickly surpasses the guide, we apply selection primarily at the beginning of training (Early AJEST) and then continue without selection; we also report a full-selection variant for completeness. At inference, we evaluate both unguided sampling and autoguided collaborative sampling.

Random data selection. As a cheap control, we uniformly sample a mini-batch of size b from each super-batch of size B, matching the filtering ratio and schedule used by selective methods. This removes learner–reference scoring entirely, yields negligible compute and memory overhead, and provides a strong time-efficiency baseline. The training loop and optimization settings are identical to those of AJEST and autoguidance; only the selection rule differs. In contrast with lowering the batch size, random data selection ensures some examples in the training dataset might never be seen by the learner model.

3. Experiments

We compare methods under two budgets: (i) equal time budget, explicitly including selection overhead; and (ii) equal data budget. This allows us to evaluate both time- and data-efficiency.

2-D Synthetic Data. We first tested our methods on Karras et al.'s 2D tree task [18]. The goal is to generate $(x;y)$ points with most of its probability density lying within a tree-shaped manifold (Figure 2). This mimics low local di-

Figure 2. Illustration of the data generation tasks. (a) Normal 2D (x, y) samples (purple points) with noise level (orange circle) are pushed towards the ground truth tree to obtain the final samples (gray points). (b) High-dimensional Normal samples are denoised to obtain natural images matching Tiny ImageNet's distribution.

Table 1. Evaluation metrics with guidance on the 2D task. Results are averaged over 5 runs with different random seeds. Standard deviation is used to indicate uncertainty. Yellow-filled cells contain the best scores. Undistinguishable results are marked in bold.

Comparison	Method	Average L2 Distance				Mandala score		Classification score	
		Full Tree		External Branches					
Same time budget	Baseline	0.246	0.297	0.613	0.210	0.73	0.11	0.94	0.01
	Early AJEST	0.110	0.019	0.516	0.019	0.75	0.08	0.93	0.02
	Full AJEST	0.118	0.025	0.514	0.020	0.53	0.12	0.85	0.04
	Random	0.101	0.006	0.508	0.009	0.71	0.08	0.93	0.01
Same data budget	Baseline	0.156	0.053	0.543	0.036	0.53	0.15	0.83	0.09
	Early AJEST	0.120	0.029	0.515	0.030	0.58	0.14	0.86	0.06
	Full AJEST	0.102	0.006	0.507	0.009	0.69	0.10	0.92	0.02
	Random	0.101	0.005	0.508	0.005	0.72	0.07	0.93	0.01

mensionality and hierarchical detail emergence in natural images [18]. We train simple diffusion models to sample from one of two classes (upper half of the tree).

We use Karras et al.'s toy model architecture [18]. We trained a small reference model for 512 iterations and a larger main learner model for 4096 iterations, optionally applying data selection with mini-batch size $b = 812$, super-batch size $B = 8192$ and filtering ratio $f = 0.8$. The loss function is evaluated on the whole batch of 8192 data points, but backpropagation is only executed for 20% of those points. Details on the model and distribution are provided in Appendix A.3. Evaluation uses loss- and coverage-based metrics (MSE, L2, mandala, and a simple classification accuracy); precise definitions are given in Appendix C.

(3×64^2)-D Image Data. We then scale up the dimensionality to Tiny ImageNet [20] using the public split (HuggingFace [38]). This dataset contains 64×64 RGB images and its training set has 200 classes following WordNet synsets as in ImageNet.

We trained an XS EDM2 model [19] as our main model for 22000 iterations. We defined a smaller XXS EDM2 model to use as the reference, and we trained it for only 5160 iterations. We used super-batch size $B = 2048$ and mini batch-size $b = 384$, effectively applying the same filtering ratio $f = 0.8$ as in the 2D toy task. Further details can be found in Appendix A.4. Evaluation uses FID [12], FD-DINOv2 [26, 32], and top-1/top-5 accuracy of samples against a pretrained Tiny ImageNet Swin-L classifier [14]; details are given in Appendix C.

4. Results

We report data-efficiency (metrics versus number of backpropagated examples) and time-efficiency (metrics versus wall-clock time, including selection overhead) as defined in Section 3. More results can be found in Appendix B.

2-D Synthetic Data. In the time-limited scenario, the 2D tree task results indicate that Early AJEST can be as time-efficient as random data selection and the baseline trained with no data selection. Despite this, Random may still be preferable due to its low algorithmic complexity and memory requirements. Results also show that Full AJEST is too time-consuming compared to other methods.

In the data-limited scenario, full AJEST performs on a similar level to random data selection, but its added complexity and time overhead makes it less preferable. Early AJEST sacrifices data-efficiency in favour of time-efficiency, but it still presents advantages over the pure autoguidance approach: it reaches slightly better results, while using less examples for backpropagation. This begets the question whether Early AJEST has the potential to unlock better performance on tasks with higher complexity and dimensionality, such as image generation.

Additional results and experimentation with other AJEST variants can be found in Appendix B.1.

(3×64^2)-D Image Data. Our Tiny ImageNet experiment shows further evidence that the time overhead of Early AJEST is negligible: it reaches results almost as good as autoguidance on this fixed time budget. AJEST proves to be inefficient from a time-based perspective for images too.

Same as in the 2D task, Early AJEST achieved similar results to autoguidance on the fixed data budget images scenario. It led to slightly better results on FID, FD-DINOv2 and Top-5 accuracy (Table 2), especially at very low data budgets (Figure 3). However, random data selection presented better perceptual metrics compared to both Early AJEST and Baseline. The computational overhead of Early AJEST makes random selection preferable.

Additional results including more generated images and other training and validation curves with all evaluated metrics can be found in Appendix B.2.

5. Discussion and limitations

Our study examines online data curation for diffusion models when combined with autoguidance. Two consistent observations emerge across tasks.

Figure 3. Training loss and validation metrics on Tiny ImageNet for EMA=0.10 and guidance weight=2.2. Curves are shown versus number of backpropagated examples (data-efficiency) and wall-clock time (time-efficiency); error bars omitted for clarity.

Table 2. Evaluation metrics on Tiny ImageNet, averaging guided single-run results from EMA=0.05 and EMA=0.10 with =1.7 and =2.2 respectively. Yellow-filled cells contain the best scores. Close results are highlighted in bold.

Comparison	Method	Perceptual metrics		Classification-based metrics	
		FID	FD-DINOv2	Top-1	Top-5
Same time budget	Baseline	30.6	624	59.8	80.6
	Early AJEST	31.0	628	58.8	79.8
	AJEST	41.0	949	31.8	54.6
	Random	27.5	699	45.4	68.8
Same data budget	Baseline	40.4	934	61.8	79.6
	Early AJEST	34.7	849	60.7	80.5
	AJEST	41.0	949	31.8	54.6
	Random	29.2	737	40.9	65.0

First, autoguidance is a strong and reliable lever for improving quality while preserving diversity. It sets a robust baseline in our experiments that is difficult to surpass with data selection alone. Early AJEST—using the guide as the JEST reference only in the beginning of training—can yield modest data-efficiency gains and reach time-to-accuracy performance close to autoguidance on both the 2D and images task. However, the dominant improvements in coverage and fidelity are driven by autoguidance.

Second, accounting for overhead matters. Full AJEST introduces significant runtime overhead and is consistently unattractive in any time-limited scenario. Early AJEST keeps the overhead negligible in our setup and can therefore be competitive when time is the primary constraint, but its advantages shrink under a fixed-data budget where autoguidance already performs strongly. Random subsetting is frequently competitive because of its near-zero complexity and memory footprint.

The following limitations should be considered when intepreting the results. (i) Scope: we evaluate on a controlled 2-D task and Tiny ImageNet at 64×64; larger resolutions, datasets, and conditionings may alter the trade-offs. (ii) Metrics: while we report FD-DINOv2 and top-1/top-5 as more rigorous alternatives to FID, definitive perceptual assessment remains challenging. (iii) Design choices: we use only one reference model, one JEST parameterization and a simple early/always selection schedule; alternative parameters, chunking, and triggers could change outcomes. (iv) Data selection: we deploy AJEST, Early AJEST and a random uniform selection baseline; other methods, as those that use small proxy models, may offer improved synergy with autoguidance and diffusion models. (v) Hyperparameters: EMA and guidance weight were not exhaustively tuned; broader sweeps may shift the relative ranking at the margins. (vi) Engineering: results reflect a single hardware/software stack; absolute runtimes and overheads may vary.

Figure 4. Qualitative results for fixed time budget on the (a) synthetic 2-D and (b) image generation tasks. For best viewing and fixed data budget results, please see full page figures in Appendix B.

Taken together, these results suggest a practical recipe: rely on autoguidance as the main driver of sample quality and diversity, and layer Early AJEST when training-time is scarce or when modest gains in data-efficiency are valuable relative to its small additional complexity. More aggressive selection schedules or heavier scoring generally do not justify their overhead in the setting explored here.

6. Conclusions

We evaluated Autoguided JEST (AJEST) for diffusion models on a 2-D toy task and Tiny ImageNet 3 at 64^2 under equal time and data budgets. Autoguidance consistently improved quality and diversity and serves as a strong baseline. Early AJEST provided small but measurable data-efficiency gains with a minimal time overhead. In practice, we recommend autoguidance as the default, with Early AJEST added when training-time is constrained or when minor data-efficiency gains justify the added machinery. Future work includes exploring richer selection scores and schedules, larger and higher-resolution datasets, and broader metric suites.

References

[1] Sharat Agarwal, Himanshu Arora, Saket Anand, and Chetan Arora. Contextual diversity for active learning. In Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVI, page 137–153, Berlin, Heidelberg, 2020. Springer-Verlag. 1

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse, uncertain gradient lower bounds. In International Conference on Learning Representations, 2020. 1

[3] Bernd Bischl, Giuseppe Casalicchio, Taniya Das, Matthias Feurer, Sebastian Fischer, Pieter Gijsbers, Subhaditya Mukherjee, Andreas C. Müller, László Németh, Luis Oala, Lennart Purucker, Sahithya Ravi, Jan N. van Rijn, Prabhant Singh, Joaquin Vanschoren, Jos van der Velde, and Marcel Wever. Openml: Insights from 10 years and more than a thousand papers. Patterns, 6(7):101317, 2025. 1

[4] Rania Briq, Jiangtao Wang, and Stefan Kesselheim. Data pruning in generative diffusion models, 2025. 1

[5] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 2818–2829, 2023. 1

[6] Cody Coleman, Christopher Yeh, Stephen Mussmann, Baharan Mirzasoleiman, Peter Bailis, Percy Liang, Jure Leskovec, and Matei Zaharia. Selection via proxy: Efficient data selection for deep learning. In International Conference on Learning Representations, 2020. 1

[7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In 2009 IEEE Conference on Computer Vision and Pattern Recognition, pages 248–255, 2009. 11

[8] Talfan Evans, Nikhil Parthasarathy, Hamza Merzic, and Olivier J Henaff. Data curation via joint example selection further accelerates multimodal learning. In The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2024. 1, 2, 5

[9] Vitaly Feldman and Chiyuan Zhang. What neural networks memorize and why: Discovering the long tail via influence estimation. In Advances in Neural Information Processing Systems, pages 2881–2891. Curran Associates, Inc., 2020. 1

[10] Sachin Goyal, Pratyush Maini, Zachary C. Lipton, Aditi Raghunathan, and J. Zico Kolter. Scaling laws for data filtering– data curation cannot be compute agnostic. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 22702–22711, 2024. 1

[11] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In Advances in Neural Information Processing Systems. Curran Associates, Inc., 2018. 1

[12] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In Advances in Neural Information Processing Systems. Curran Associates, Inc., 2017. 3, 10

[13] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance, 2022. 1, 2

[14] Ethan Huynh. Vision transformers in 2022: An update on tiny imagenet, 2022. 3, 10, 11

[15] Lu Jiang, Zhengyuan Zhou, Thomas Leung, Li-Jia Li, and Li Fei-Fei. MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels. In Proceedings of the 35th International Conference on Machine Learning, pages 2304–2313. PMLR, 2018. 1

[16] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. 1

[17] Tero Karras, Miika Aittala, Samuli Laine, and Timo Aila. Elucidating the design space of diffusion-based generative models. In Proceedings of the 36th International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 2022. Curran Associates Inc. 2, 1

[18] Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. Guiding a diffusion model with a bad version of itself. In Advances in Neural Information Processing Systems, pages 52996–53021. Curran Associates, Inc., 2024. 1, 2, 3, 5, 10

[19] Tero Karras, Miika Aittala, Jaakko Lehtinen, Janne Hellsten, Timo Aila, and Samuli Laine. Analyzing and improving the training dynamics of diffusion models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 24174–24184, 2024. 2, 3, 10

[20] Ya Le and Xuan S. Yang. Tiny imagenet classification with convolutional neural networks. Course Project Report Winter Quarter 2015, Stanford University, CS231n: Convolutional Neural Networks for Visual Recognition, 2015. Unpublished student project, available online. 3, 2, 11

[21] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pages 9992–10002, 2021. 10

[22] Baharan Mirzasoleiman, Jeff Bilmes, and Jure Leskovec. Coresets for data-efficient training of machine learning models. In Proceedings of the 37th International Conference on Machine Learning, pages 6950–6960. PMLR, 2020. 1

[23] mnmoustafa and Mohammed Ali. Tiny imagenet (mnmoustafa version) on kaggle. https://kaggle.com/competitions/tiny-imagenet, 2017. Accessed: 2025-08-24. 11

[24] Luis Oala, Manil Maskey, Lilith Bat-Leah, Alicia Parrish, Nezihe Merve Gürel, Tzu-Sheng Kuo, Yang Liu,

Rotem Dror, Danilo Brajovic, Xiaozhe Yao, Max Bartolo, William A Gaviria Rojas, Ryan Hileman, Rainier Aliment, Michael W. Mahoney, Meg Risdal, Matthew Lease, Wojciech Samek, Debojyoti Dutta, Curtis G Northcutt, Cody Coleman, Braden Hancock, Bernard Koch, Girmaw Abebe Tadesse, Bojan Karlaš, Ahmed Alaa, Adji Bousso Dieng, Natasha Noy, Vijay Janapa Reddi, James Zou, Praveen Paritosh, Mihaela van der Schaar, Kurt Bollacker, Lora Aroyo, Ce Zhang, Joaquin Vanschoren, Isabelle Guyon, and Peter Mattson. DMLR: Data-centric machine learning research - past, present and future. *Journal of Data-centric Machine Learning Research*, 2024. 1

[25] Patrik Okanovic, Roger Waleffe, Vasilis Mageirakos, Konstantinos Nikolakakis, Amin Karbasi, Dionysios Kalogerias, Nezihe Merve Gürel, and Theodoros Rekatsinas. Repeated random sampling for minimizing the time-to-accuracy of learning. In *The Twelfth International Conference on Learning Representations*, 2024. 1, 3

[26] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel HAZIZA, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. Featured Certification. 3, 10, 11

[27] Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. Deep learning on a data diet: Finding important examples early in training. In *Advances in Neural Information Processing Systems*, pages 20596–20607. Curran Associates, Inc., 2021. 1

[28] Omead Pooladzandi, David Davini, and Baharan Mirzasoleiman. Adaptive second order coresets for data-efficient machine learning. In *Proceedings of the 39th International Conference on Machine Learning*, pages 17848–17869. PMLR, 2022. 1

[29] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations*, 2018. 1

[30] Seungjae Shin, Heesun Bae, Donghyeok Shin, Weonyoung Joo, and Il-Chul Moon. Loss-curvature matching for dataset selection and condensation. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, pages 8606–8628. PMLR, 2023. 1

[31] Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari Morcos. Beyond neural scaling laws: beating power law scaling via data pruning. In *Advances in Neural Information Processing Systems*, pages 19523–19536. Curran Associates, Inc., 2022. 1

[32] George Stein, Jesse Cresswell, Rasa Hosseinzadeh, Yi Sui, Brendan Ross, Valentin Villecroze, Zhaoyan Liu, Anthony L Caterini, Eric Taylor, and Gabriel Loaiza-Ganem. Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models. In *Advances in Neural Information Processing Systems*, pages 3732–3784. Curran Associates, Inc., 2023. 3, 10

[33] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016. 10

[34] Haoru Tan, Sitong Wu, Fei Du, Yukang Chen, Zhibin Wang, Fan Wang, and Xiaojuan Qi. Data pruning via moving-one-sample-out. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2023. Curran Associates Inc. 1

[35] Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations*, 2019. 1

[36] Vishaal Udandarao, Nikhil Parthasarathy, Muhammad Ferjad Naeem, Talfan Evans, Samuel Albanie, Federico Tombari, Yongqin Xian, Alessio Tonioni, and Olivier J. Hénaff. Active data curation effectively distills large-scale multimodal models, 2025. Accepted for the IEEE/CVF Conference on Computer Vision and Pattern Recognition 2025 to be hosted in June 2025. 1, 2

[37] Xiaobo Xia, Jiale Liu, Jun Yu, Xu Shen, Bo Han, and Tongliang Liu. Moderate coreset: A universal method of data selection for real-world data-efficient deep learning. In *The Eleventh International Conference on Learning Representations*, 2023. 1

[38] zh plus. Tiny imagenet (zh-plus version) on hugging face. https://huggingface.co/datasets/zh-plus/tiny-imagenet, 2025. Accessed: 2025-08-19. 3, 2, 11

Autoguided Online Data Curation for Diffusion Model Training

Supplementary Material

A. Implementation details

A.1. JEST data selection

Joint example selection (JEST) samples training examples based on a learnability score [8]. Assuming that datapoints with indices $i \ 2 \ 1; :::; B$ are fed into the learner model and samples with indices $j \ 2 \ 1; :::; B$ are fed into the reference model, the learnability score s_{ij} is defined as...

$$s^{learn}_{ij} = L^{L}_{ij} \quad L^{R}_{ij} \tag{1}$$

Where L^L is the loss evaluated by the learner model and L^R is the loss evaluated by the reference model.

The joint batch selection is done in an iterative process with N steps by selecting n chunks of size $\frac{b}{N}$ (Figure 5). The first chunk C_0 is populated sampling from a uniform probability distribution over all B datapoints. For each successive chunk C_n, sampling is done on a conditional probability distribution over the B datapoints given all $n\frac{b}{N}$ previously selected datapoints that define $K = C_1 \ [\ ::: \ [\ C_n$ set. This conditional probability distribution is modeled as a softmax distribution that takes certain logits as input. The logits $z = (z_1; :::; z_B)$ are calculated adding four vector terms of length B.

1. The $(s_{11}; :::; s_{BB})$ diagonal scores that feed the same datapoint to both the learner and the reference models.
2. The sums of scores $(\overset{P}{}_{k2K} s_{k1}; :::; \overset{P}{}_{k2K} s_{kB})$ that result from only considering the selected datapoints fed into the learner model.
3. The sums of scores $(\overset{P}{}_{k2K} s_{1k}; :::; \overset{P}{}_{k2K} s_{Bk})$ that result from only considering the selected datapoints fed into the reference model.
4. A penalizing term whose elements are 10^8 for all selected datapoints in K and 0 for unselected datapoints.

Figure 5. Iterative sampling process for JEST data selection.

Our code implementation follows quite closely the one published by Evans et al [8]. However, we observed that a softmax distribution applied directly to these z logits is highly unstable. For positive increasing scores, the exponential of the logits is unbounded and quickly causes a numerical overflow. We therefore applied a normalization factor to the logits before using them as input to the softmax function. We noticed that at any given JEST iteration with index n, the logits are calculated as the sum of $n\frac{b}{N}$ terms. To bound the logits during the full iterative process, we simply divided them by that number, resulting in the following final expression:

$$z_l = \frac{N}{nb} \quad s_{ll} + \underset{k2K}{X} s_{kl} + \underset{k2K}{X} s_{lk} \qquad l \tag{2}$$

A.2. Autoguidance and hyperparameters

By Karras et al.'s terminology [18], diffusion models are used to evolve any noisy sample $x_{initial} \quad p(x; \ _{max})$ from a Gaussian distribution $N(x; \ ^2 I)$ with maximum noise level $_{max}$ to a sample matching the ground truth distribution $x_{final} \quad p(x; 0)$ with zero noise. The following probability flow ordinary differential equation needs to be solved:

$$dx = \quad S(x; \ ) \, d \tag{3}$$

$$S(x; \ ) = r_x \ \log p(x; \ ) \tag{4}$$

The diffusion model is trained to predict the score for every $\ 2 \ [0; \ _{max}]$ and every example $x \quad p(x; \ )$. A $\ = \ t$ noise schedule set by a time step t can be used to obtain the final sample using an iterative approach [17]. Autoguidance improves the performance of diffusion models replacing the score from Eq. (3) with an interpolation of scores defined as in Eq. (5):

$$S = \ S^{main} + (1 \quad)S^{ref} \tag{5}$$

The S^{main} score is evaluated by the main model and the S^{ref} score is evaluated by an auxiliary reference model. The two models share the same architecture and conditioning, but the reference model is assumed to be a weaker version of the main model: a smaller model that is trained for less iterations. This applies a corrective force that pushes samples on each step of the denoising process towards regions on which the reference and the main model disagree on their predictions. This guidance is beneficial because it acts on the assumption that both the reference and the main model will make similar mistakes.

The guidance weight is a key hyperparameter in our experiments. Autoguidance influences the results for any $\ > \ 1$. If set to $\ = 1$, the main scores are retrieved; if set to $= 0$, the reference scores are retrieved instead. Another key hyperparameter is the EMA length: the duration over which an Exponential Moving Average (EMA) on the models weights is calculated during training.

Whenever available, we useand EMA values directly extracted from Karras et al.'s publication [18]. For the 2D tree task, we use their default value $= 3$. On ImageNet-64 with EDM-XS, Karras et al report the best FID metric for $= 1:7$ and EMA=0.045 and the best FD-DINOv2 metric for $= 2:2$ and EMA=0.105. We train with default EMA=0.100 and EMA=0.050 values, so we adopt $= 2:2$ for the first one and $= 1:7$ for the second one.

A.3. 2D tree data generation task

The ground truth distribution for this task is modeled with a mixture of multi-variate 2D Gaussians (see Figure 6). Each brach is created out of 8 Gaussians uniformly-distributed along its central line segment. The exact same parameters from Karras et al [18] were used. The upper half of the tree is assigned to class A -the class we aim to sample from- and the lower half of the tree is assigned to class B. The tree is designed to have depth level 7, meaning a branch is split in two 7 times. To gain further resolution on our metrics and data analysis, we define as "external branches" all those branches with depth level 5 or more.

As described in [18], we built 2D diffusion models following a simple multi-layer perceptron architecture. The neural network's input consisted of a set of (x, y) coordinates and the noise level that correlates with the time step t of the denoising process. Four hidden layers with the same number of neurons followed the input layer. The scalar output was trained to represent the logarithm of the unnormalized probability density. Differentiating this value, the model could also return the score function. Training was done via exact score-matching. The loss function was defined as the mean squared error between the score and the ground truth score scaled by a ^{2} factor (see Equation (7) in C).

Figure 6. Visualization of the 2D tree ground-truth distribution. (a) A contour of the 2D tree distribution, showing which areas are considered class A and its external branches. (b) Color map of the logarithm of the ground-truth probability distribution function $\log p$. (c) A mixture of Gaussians is used to model the tree distribution and its peaks are visible in the 3D plot of $\log p$.

Following Karras et al. [18], we first trained a small reference model for only 512 iterations with hidden dimension 32. We then independently trained a larger main model with hidden dimension 64 for 4096 iterations. We trained on a single NVIDIA A5000 GPU. In this setup, a no-selection baseline could be trained for 4096 iterations in approximately 23 minutes; Full AJEST increased this to 36 minutes, while Early AJEST added only 38 seconds over baseline.

We store models every 128 training iterations and we evaluate data and time efficiency at the latest point possible during training. For the equal wall-time scenario, this happens at the last epoch of Baseline, the fastest method; the closest matching checkpoints are collected from all other methods, resulting on an average training time of $(16:6 \quad 0:5)$ min. For the scenario with equal number of backpropagated examples, this happens at the last epoch of AJEST and Random, the methods with the highest number of filtered examples; closest matching checkpoints result on an average of $(3:3 \quad 0:2)$ million examples.

A.4. EDM2 for Tiny ImageNet

As Karras et al. [18], we used the EDM2 diffusion model architecture [19] for image generation. EDM2 diffusion models are based on a UNet network with 4 blocks on its decoder and encoder. The XS EDM2 model has 128 channels on its first UNet block, whereas the XXS EDM2 model desgined by us has 64 channels instead. We trained these models to solve a natural images generation task on Tiny ImageNet [20] as found in HuggingFace [38]. The training dataset contains 500 $64 \quad 64$ RGB images for each of its 200 classes, adding up to a total of 1 million training examples. The categories correspond to synsets of the WordNet hierarchy, same as they do in the widely-used ImageNet dataset.

We know that Karras et al. [18] trained the XS model with batch size 2048 until it had seen 2147.5 Mimg from ImageNet, a dataset containing 1,281,167 training examples. We therefore estimated we would need to train the XS model for 167.6 Mimg in Tiny ImageNet to achieve convergence, assuming linear scaling on the training dataset size out of simplicity. Karras et al. [18] used a smaller model trained for 1/16th training iterations as their reference, so we decided we could use an XXS EDM2 model trained for 5160 iterations instead of the 81845 iterations required to go through 167.6 Mimg at batch size 2048.

We executed parallel-data distributed training on 2 NVIDIA A5000 GPUs. However, our dual GPU system could only achieve a speed of 8.2 sec/epoch or 1.1 hs/Mimg, forcing the convergence of the XS model to require 7.7 days. As we had seen that the best results in the 2D task could only be achieved by early data selection, we decided to stop training the XS main model at 48 hs and 22000 iterations. However, we kept the reference model trained for 5160 iterations. In consequence, our reference model was trained for a 1/4th of the main model's total iterations, 4 times more what Karras et al do.

We converted the 8-bits Tiny ImageNet images into $[-1, 1]$ float images, and applied no other normalization besides the input preconditioning from the EDM2 model. We used Adam optimizer and we adopted Karras et al's EDM2 learning rate schedule [19]. However, to train with data selection we follow Okanovik et al's recommendation [25] on how to adapt the learning rate schedule to decay as fast as it would when training on the entire dataset (see Figure 1 from [19]).

We stored models every 500 training iterations. We followed the same methodology as in the 2D tree task to select a fix data and time budget for our analysis. For the equal wall-time scenario, this corresponded to (47.73 ± 0.07) hs; for the scenario with equal number of backpropagated examples, (7.2 ± 0.2) Mimg.

B. Additional results

B.1. 2D toy example with all metrics evaluated

Results for the complete set of metrics evaluated both with guidance and no guidance at equal wall time and equal number of backpropagated examples are presented on Table 3. We also include visual representations of the distribution of generated points in Figures 7 and 8 for the fixed time and fixed data budget scenarios, both with guidance and without it.

Unguided models reach better performance on the average L2 metric in external branches, but this is in clear conflict with the perceptual quality assessment that can be done on the generated points distributions. As Karras et al. state, guided models generate points closer to the high-density regions of the distribution, avoiding regions between branches where the ground-truth probability density is low. This is what originally led us to develop the classification and mandala scores (see Appendix C), a set of metrics that better reflects autoguidance advantages and rewards the desired behaviour of diffusion models in the 2D toy task.

B.2. EDM2 for Tiny ImageNet

We present images generated on the fix-budget and fix-time scenarios in Figure 9. In the same-time comparison, it is remarkable for Early AJEST to lead to the first human face and the first goldfish with eyes. The autoguidance baseline, however, leads to the first tractor and the closest image to a golden retriever's face.

We also present images generated using different EMA and guidance weight in Figure 10. These hyperparameters are shown to have a large influence on the image quality, as first reported by Karras et al. [18]. Guidance improves sharpness and contents considerably for EMA=0.10, but not as much for EMA=0.05. This might be due to 2.2 being closer to the ideal EMA for 0.10 than 1.7 is for 0.05.

Tables with separate results for EMA=0.05 and EMA=0.10 -both with guidance and without it- can be seen in Tables 5 and 5. Results averaged over those two EMA, but with both guided and unguided metrics, are presented in Table 6.

Our Tiny ImageNet results show new evidence that autoguidance is especially beneficial to improve coverage and precision over the ground-truth distribution: guidance improves all Swin-L top-1 and top-5 results, for as much as 10% in the best-case scenario.

In the time-limited scenario, there is a marked difference between FID and all other metrics. Random reaches the best FID score, and Early AJEST outperforms autoguidance. However, all other metrics show Baseline as the best, followed by Early AJEST, Random and finally AJEST. This may be added to the list of alarming evidence on how FID might not be an adequate metric to measure generated image quality [32]. We strongly believe that generative vision AI would benefit from better metric definitions.

We include training and validation curves for different EMA and guidance combinations in Figures 11 to 14. The trigger for Early AJEST can be deduced from the training curves, as its training loss falls from AJEST's to Baseline's. Data selection methods lead to consistently better results at low data budgets, especially Full AJEST and random data selection. Early AJEST only shows limited-data advantages over Baseline on FID and FD-DINOv2, but it follows quite closely its Top-1 and Top-5 performance under limited-time settings.

Evidence suggests that data selection might help mitigate overfitting on FID and FD-DINOv2 for unguided models: this is especially noticeable on the FID plot for EMA=0.05 and $= 1.7$ (Figure 14). However, this effect might also be explained by this being a very early stage in training for AJEST and Random; the same overfitting phenomenum might be present at later training times.

B.3. 2D toy example and other AJEST strategies

We explored the possibility of using both JEST's learnability score and an inverted learnability score:

$$s_{ij}^{inv} = L_{ij}^{R} - L_{ij}^{L} = -s_{ij}^{learn} \tag{6}$$